.NET Program Execution Process

Back to: C.NET Tutorials For Beginners and Professionals

.NET Program Execution Process

In this article, I will discuss the **.NET Program Execution Process Flow** in Detail. Please read our previous article discussing the Common Language Runtime (CLR) Architecture in detail. As .NET Developers, we should know when we create an application, how the application is compiled and how the .NET Framework executes the application. But before understanding the **.NET Program Execution** process, let us first understand how non-dot net applications such as C, VB, and C++ programs are executed.

Non-DOT NET Program Execution Process:

We know that computers only understand machine-level code. The Machine-level code is also known as native code or binary code. So when we compile a C, VB6, or C++ program, the language compiler compiles the language source code. It generates the native machine code (also called binary code), which can be understood by the underlying operating system. The general flow of the process is shown in the below image.

The Native code or machine code that is generated by the respective language compiler is specific to the operating system on which it is generated. If we take the compiled native code and try to run it on another operating system, then it will fail. So the problem with this type of program execution is that it is not portable from one platform to another. That means it is platform-dependent.

.NET Program Execution Process:

Now let us understand the .NET Program Execution process in detail. Using .NET, we can create different types of applications such as Console, Windows, Web, and Mobile Applications, irrespective of the type of application, when we execute any .NET application, the following things happen in order.

The .NET Application Source Code gets compiled into Microsoft Intermediate Language (MSIL), which is also called Intermediate language (IL) or Common Intermediate language (CIL) code. Both .NET and Non-DOT.NET applications generate an assembly when we compile the application. Generally, the assemblies have an extension of .DLL or .EXE based on the type of application we compile. For example, if we compile a Windows or Console application in .NET, we get an assembly of type EXE, whereas when we compile a Web or Class Library Project in .NET, we get an assembly of type .DLL.

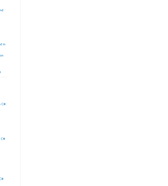
The difference between a .NET and NON-DOTNET assemblies is that .NET Assembly is an Intermediate Language format, whereas the NON-DOTNET assembly is a native format.

The NON .NET applications can run directly on top of the operating system as the NON-DOTNET assembly contains the native code, whereas .NET applications run on top of a virtual environment called **Common Language Runtime (CLR)**. It contains a component called **Just-In-Time Compiler (JIT)** which will convert the Intermediate language into native code, which can be understood by the underlying operating system.

.NET Program Execution Steps:

In .NET, the application execution consists of 2 steps. They are as follows.

In the step1 the respective language compiler compiles the Source Code into Intermediate Language (IL). In the 2nd step, the JIT compiler (CLR) will convert the Intermediate Language (IL) code into native code (Machine Code or Binary Code), which the underlying operating system can then execute. The above process is shown in the image below.



As the .NET assembly is in Intermediate Language (IL) format and not in native or machine code, the .NET assemblies are portable to any platform as long as the target platform has the **Common Language Runtime (CLR)**. The target platform's CLR converts the Intermediate Language code into native or machine code that the underlying operating system can understand.

Intermediate Language code is also called managed code. This is because CLR manages the code that runs inside it. For example, it is VB program, the developer is responsible for de-allocating the memory consumed by an object. If a developer forgets to de-allocate the memory, then it will result in memory leaks and will slow down the PC. But managed code does not need to worry about de-allocating the memory consumed by an object. Automatic memory management also known as garbage collection is provided by CLR. Apart from garbage collection, the CLR provides several other benefits, which we will discuss in a later session. Since CLR manages and executes the Intermediate Language (IL), it is also called the managed code.

.NET supports different programming languages like C#, VB, JIT and C++. C#, VB, and JIT can only generate managed code (IL), whereas C++ can generate both managed code (IL) and unmanaged code (Native code).

The native code is not stored permanently anywhere after we close the program the native code is thrown away. When we execute the program again, the native code is generated again.

The .NET program is similar to the Java program's execution. In Java, we have bytecodes and JVM (Java Virtual Machine) whereas in .NET we have Intermediate Language and CLR (Common Language Runtime).

In the next article, I will discuss the **Intermediate Language (IL) Code** in C# with examples. In this article, I try to explain the DOT.NET Program Execution process. I hope you enjoy the article.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3000+ articles over the last 11 year career. Pranaya Rout has very good experience with Microsoft Technologies including C#, VB, ASP.NET MVC, ASP.NET Web Forms, .NET Core, SQL Server, .NET Core, LINQ, EF Core, WPF/SL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Previous Lesson
Common Language Runtime in .NET Framework

Next Lesson
Intermediate Language (CIL)/(IL) & (L)ANG Code in C#

5 thoughts on "NET Program Execution Process"

SUNITA SINHA
OCT 9, 2019 AT 12:40 AM
Thanks! Clear explanations make easy understanding.
Reply
SHAMIULA
SEPTEMBER 24, 2020 AT 5:25 AM
Love your article. Make now
Reply
AMIT
MAY 5, 2020 AT 7:35 AM
Excellent!!!
Reply
SUNNA ANSARI
JANUARY 12, 2021 AT 8:00 AM
Thank you for your article ……
Reply
TIBU
AUGUST 12, 2020 AT 2:57 PM
really good
Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *
Email *
Website

Post Comment

Online Training
C.NET Online Training Program
ASP.NET Core Training
Microservices Online Training Program
Microsoft Azure Training
Angular Training
Introduction & Environment Setup
Free Computer Books
Introduction to Programming Language
Free Computer Programs
Free Tools
Different Types of Applications
Programming Methodologies
Algorithm, Pseudocode, Flowchart, Programs
Introduction to .NET Framework
.NET Framework Architecture and Components
Introduction to C# Programming Language
How to Download and Install Visual Studio on Windows
Coding First Console Application using Notepad
.NET Developer Roadmap for 2021
Coding Standard Best Practices
C# NET Basics
Data Types in C#
Methods and Properties of Console Class in C#
Data Types in C#
Variables in C#
Type Casting in C#
Operators in C#
Operators in C#
Control Flow Statements in C#
Loops in C#
While Loop in C#
Do While Loop in C#
For Loop in C#
Switch Statements in C#
Goto in C#
While Loop in C#
Do While Loop in C#
For Loop in C#
Continue Statement in C#
Continue Statement in C#
Goto Statement in C#
Functions in C#
User Defined Functions in C#
Passing Parameters by Reference in C#
Recursion in C#
Char Input and Output in C#
Command Line Arguments in C#
Array in C#
Array Length in C#
Array Length in C#
Jagged Array in C#
Jagged Array in C#
Array Class Properties and Methods Interview Questions in C#
How to use Abstract Classes and Methods in C#
Abstract Class and Abstract Methods Interview Questions in C#
How to use Abstract Classes and Methods in C#
Interfaces in C#
Interfaces Interview Questions and Answers in C#
Interfaces in C# Example
Difference between Interfaces and Abstract Classes in C#
Implicit Interfaces in C#
Explicit Interfaces in C#
Interfaces in C#
Interfaces in C#
Type of Inheritance in C#
How to use Interfaces for Application Development
Interfaces Interview Questions in C#
Abstract Class and Abstract Methods Interview Questions in C#
How to use Abstract Classes and Methods in C#
Sealed Class and Sealed Methods in C#
Extension Methods in C#
Static Class in C#
Variable Reference and Instance of a Class in C#
OOPs Real Time Examples
Real Time Examples of Encapsulation Principle in C#
Real Time Examples of Abstraction Principle in C#
Real Time Examples of Inheritance Principle in C#
Real Time Examples of Polymorphism Principle in C#
Real Time Examples of Interface in C#
Real Time Examples of Abstract Class in C#
Events, Delegates and Lambda Expressions in C#
Course Structure of Events, Delegates and Lambda Expressions
Introduction to Events in C#
Rules of Events, Delegates and Event Handler in C#
Delegates in C#
Multicast Delegates in C#
Delegate Real Time Example in C#
Generic Delegates in C#
Anonymous Method in C#
Lambda Expressions in C#
Events in C# with Example
Multi-Threading
Multithreading in C#
Thread Class in C#
How to Pass Data to Thread Function in Type Safe Manner in C#
How to Retrieve Data from a Thread Function in C#
Join Method and Thread Property of Thread Class in C#
Thread Synchronization in C#
Lock in C#
Critical Section in C#
Mutex Class in C#
Semaphore Class in C#
SemaphoreSlim Class in C#
Deadlock in C#
Performance Testing of a Multi-Threaded Application
Thread Pool in C#
Foreground and Background Threads in C#
AutoResetEvent and ManualResetEvent in C#
Thread Life Cycle in C#
Thread Priorities in C#
How to Terminate a Thread in C#
Use Thread Communication in C#
How to Debug a Multi-threaded Application in C#
Collections in C#
Array in C#
2D Array in C#
Advantages and Disadvantages of Array
Collections in C#
Arraylist in C#
Hashtable in C#
Stack Generic Stack in C#
Stack Generic Queue in C#
Stack Generic Queue in C#
Advantages and Disadvantages of Non-Generic Collections in C#
Generic Collections in C#
Generic Stack in C#
Generic Queue in C#
Generic Collections in C#
How to Sort a List of Complex Type in C#
Generic Dictionary in C#
Dictionary Collections Class in C#
Conversion Between Array, List and Dictionary in C#
Generic SortedList in C#
Generic Sorted Dictionary Class in C#
Generic Sorted Collection Class in C#
Generic SortedSet Collection Class in C#
Generic LinkedList Collection Class in C#
Generic HashSet Collection Class in C#
ConcurrentDictionary Collection Class in C#
ConcurrentQueue Collection Class in C#
ConcurrentStack Collection Class in C#
ConcurrentBag Collection Class in C#
BlockingCollection in C#
File Handling
File Handling in C#
FileStream Class in C#
Encryption and Decryption in C#
StreamWriter in C#
File Class in C#
TextWriter and TextReader in C#
StreamWriter and BinaryReader in C#
StringWriter and StringReader in C#
Filehttp Class in C#
DirectoryInfo Class in C#
Reader and Import Excel Data in C#
Serialization
Asynchronous Programming
Introduction to Concurrency, Async and Await in C#
Task in C#
How to Return a Value from Task in C#
Task Cancellation in C#
How to Limit Number of Concurrent Tasks in C#
How to Cancel a Task in C#
Task Continuation in C#
How to Create Synchronous Method using Task in C#
Retry Pattern in C#
Only One Pattern in C#
Task Cancellation Pattern in C#
Task in C#
Task Based Asynchronous Programming in C#
Chaining Tasks by Using Continuation in C#
How to Attached Child Tasks in a Parent Task in C#
DataFlow in C#
How to Cancel a Non-Cancelable Task in C#
Asynchronous Streams in C#
How to Cancel Asynchronous Stream in C#
Parallel Programming
Task Parallel Library in C#
Parallel For in C#
Parallel.Invoke Loop in C#
Parallel Foreach in C#
Parallel Aggregation in C#
Parallelism in C#
How to Cancel Parallel Operations in C#
Monitor, Mutexlock, Threads and Shared Conditions in C#
Interlocked.xx Lock in C#
Parallel.ORDER in C#
Multithreading in Windows Forms Programming in Parallel Programming in C#
AutoMapper
AutoMapper in C#
AutoMapper Complex Mapping in C#
How to Map Complex Type to Primitive Type in AutoMapper in C#
AutoMapper Reverse Mapping in C#
AutoMapper Conditional Mapping in C#
AutoMapper Ignore Method in C#
Flat and Dynamic Values in Destination Property in AutoMapper
Optional Parameters, Indexers and Enum
How to Make Optional Parameters in C#
Indexers in C#
Indexers Real Time Example in C#
Enums in C#
.NET Framework Architecture
DOT NET Framework Architecture and Common Language Runtime in .NET Framework
.NET Program Execution Process
Common Language Runtime (CLR) Code in C#
Common Type System in .NET Framework
Common Language Specification in .NET Framework
Managed and Unmanaged Code in .NET Framework
Boxing Vs. ESS in .NET Framework
Key Features in .NET Framework
Driving and Weak Assemblies in .NET Framework
Strong Name Assembly (SN.EXE) in .NET Framework
DLL Hell Problem and Solution in .NET Framework
Var, Dynamic and Reflection
Reflection in C#
Dynamic Type in C#
Var Keyword in C#
Var in Dynamic in C#
Dynamic vs Reflection in C#
Anonymous Type in C#
Var vs Var in C#
Named Parameters in C#
C# 7.0 new Features
C# 7.0 Tuple Features
Enhancement in Out Variables in C# 7.0
Pattern Matching in C#
Digit Separators in C# 7
Tuples in C# 7
Splitting Tuples in C# 7
Tuple Deconstruction in C#
Ref Returns and Ref Locals in C# 7
Constrained Runtime Member Types in C#
Generic Async Binded Members in C#
Throw Expression in C#
Async Main in C#
C# 8 New Features
C# 8 New Features
Readonly Structs in C#
Default Interface Methods in C#
Indexers in C#
Pattern Matching in C#
Using Declarations in C#
Static Local Functions in C#
Disposable Ref Structs in C#
Nullable Reference Types in C#
Asynchronous Streams in C#
Asynchronous Disposable in C#
Indices and Ranges in C#
Null Coalescing Assignment in C#
Enhancement in Unmanaged Constructed Types in C#
Stackalloc in C#
Most Popular C# Books
Most Recommended C# Books
Most Recommended Data Structure and Algorithm Books